Sign up to discover human stories that deepen your understanding of the world. Become a Medium member for unlimited reading. Enjoy curated stories, ad-free experience, and more benefits. Upgrade for $5/month.
Upgrade

GenAI — Finetuning OpenSource LLaVA-NeXT

Rajesh Naroganeni (Raj) · Follow
Published in Raj Naroganeni (Raj) · Follow
7 min read · Nov 18, 2024

A step by step guide to fine-tune llava-next VLM for specific use case with code snippets.

#gcloud #vllm #llamafactory #llava-next #vision

Table of Contents:

1. Introduction.
2. How Does Text Generation Happens in VLMs.
3. LLaVa Next Architecture.
4. Why Fine-tuning of Open-Source?
5. Fine-tuning of VLMs intuition.
6. LLaVa Next Finetuning Code.

Introduction:

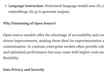
Vision-Language Models (VLMs) are a subset of AI models that integrate visual and text data to perform tasks such as image captioning, visual question answering, and image segmentation. These models are crucial in bridging the gap between visual and language understanding, enabling more intuitive interactions.

Understanding Vision-Language Models (VLMs):

Source: Image Reproduced from Author

- **Transformer Language Models:** Underlined complex language structures.
- **Conditional Approach:** Enhance understanding of relationships within and between image objects.
- **Visual ML Algorithms:** Analyze patterns and relationships in images.

How Does Text Generation Happens in VLMs:



Source: Image Reproduced from Author

- **Parallel Streams:** Image and text processed separately via embedding and self-attention.
- **Cross-Attention:** Aligns visual features with text tokens for context.
- **Joint Representation:** Captures relationships between image regions and text.
- **Text Generation:** Produces responses based on visual linguistic understanding.

LLaVa Next Architecture:

Figure 1: LLaVA network architecture.

Source: LLaVA Paper

- **Vision Encoder:** Processes image (X_v) to generate visual features (Z_v).
- **Projection Layer:** Maps Z_v to higher dimensional vision embeddings (H_v), aligning with the language model's latent space.
- **Language Instruction:** Pretrained language model uses (H_v) and text embeddings (H_q) to generate outputs.

Why Fine-tuning of Open-Source?

Open-source models offer the advantage of accessibility and community-driven improvements, making them ideal for experimentation and customization. In contrast, enterprise models often provide robust support and optimized performance but may come with higher costs and less flexibility.

Data Privacy and Security

- **Internal Hosting:** Open-source models can be hosted internally, allowing organizations to maintain control over sensitive data and ensure compliance. This is crucial for industries like healthcare and finance, where data privacy is paramount.
- **Control Over Data:** By fine-tuning models in-house, organizations can ensure that their proprietary data is not exposed to third-party services, reducing the risk of data breaches.

Transparency and Trust

- **Access to Source Code:** With open-source models, organizations have full access to the source code, allowing them to understand the model's workings and ensure there are no hidden functionalities or biases.
- **Auditable and Modifiable:** Organizations can audit the code for security vulnerabilities and modify it to align with their ethical standards and business requirements.

Scalability and Experimentation

- **Experimentation:** Open-source models allow for extensive experimentation without the constraints of vendor-imposed limitations, enabling organizations to explore new ideas and approaches.
- **Scalability:** Organizations can scale their solutions as needed, without being tied to the limitations or pricing structures of enterprise models.

Fine-tuning of VLMs

One subscription. Endless stories.
Become a Medium member for unlimited reading.
Upgrade now

We use LoRA + QLoRA :-

Source: LoRA Paper

Benefits of LoRA:

- LoRA weights can be summed with pretrained weights
- Total # of parameters doesn't change
- No inference delay
- Trains **small % of total parameters**
- Only applied to attention heads
- Uses more cuda base than respective LoRA weights, but share base model weights
- **Low storage cost**

Figure 1. Illustrations of our proposed method. We only train the low-rank matrices in the attention layers for adapting the pre-trained model to multiple downstream tasks. The low-rank matrices of different tasks are then stored in a single base model.

Source: LoRA Paper

Vision Encoder Quantization:

- Reduce size of convolution or attention weights and activations.

Language Decoder Quantization:

- Self attention and feedforward layers.

Cross Attention Layer Quantization:

- In models like Llava and VILBERT, cross-attention layers are quantized.

LLaVa Next Finetuning Code:

Base Model: llava-hf/llava-v1.6-mistral-7b-hf

Make sure you have about 50-GB of GPU memory and Cuda 12.2 Installed. [Assuming your dataset is < 5000 images].

Step 1: Import Necessary Libraries:

```
import os
import sys
import torch
import datetime
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
from gpt_batch import GenerateTg
import pandas as pd
import ast
import Counter
from tqdm import tqdm
from collections import defaultdict
from sklearn.metrics import f1_score, precision_score, recall_score
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.model_selection import train_test_split
from transformers import LlavaNextForConditionalGeneration, LlavaNextProcessor, TrainingArguments, Trainer, DataCollatorForSeq2Seq
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
import bitsandbytes as bnb
import os
os.environ["CUDA_VISIBLE_DEVICES"] = "0,1,2,3"
```

Step 2: Make sure GPU is available:

```
if torch.cuda.is_available():
    print("GPU is available.")
    num_gpus = torch.cuda.device_count()
    print(f"Number of GPUs available: {num_gpus}")
    for i in range(num_gpus):
        print(f"GPU {i}: {torch.cuda.get_device_name(i)}")
else:
    print("GPU is not available.")
```

Step 3: Read the dataset.

Dataset should contain below columns:

1. **images** - binary-encoded images.
2. **category_label** - Target column.
3. **prompt** - Prompt which will be used to ask llava.

```
df = pd.read_parquet("path to the dataset")
df['category_label'].value_counts()
df['image'] = df['image'].apply(lambda x: x['bytes'])

df.sample(2)
```

Step 4: Convert Prompt to match model input format:

```
# function to convert each row to the specified ChAT format
def convert_to_chat(row):
    return [
        {
            "role": "system",
            "content": [
                {
                    "type": "text",
                    "text": "You are a helpful assistant."
                }
            ]
        },
        {
            "role": "user",
            "content": [
                {
                    "type": "image",
                    "text": null,
                    "index": 0
                },
                {
                    "type": "text",
                    "text": row['prompt']
                }
            ]
        },
        {
            "role": "assistant",
            "content": [
                {
                    "type": "text",
                    "text": row['category_label'].lower()
                }
            ]
        }
    ]

# Apply the function to each row in the DataFrame
df['chat_messages'] = df.apply(convert_to_chat, axis=1)
```

Step 5: Load the base model from huggingface:

We quantize the model to 4bit.

```
model_id = "llava-hf/llava-v1.6-mistral-7b-hf"

quantization_config = BitsAndBytesConfig(
    load_in_4bit=True
)

model = LlavaNextForConditionalGeneration.from_pretrained(model_id,
                                          quantization_config=quantization_config,
                                          torch_dtype=torch.float16)
```

Step 6: Applying LLava Chat Template:

```
LLAVA_CHAT_TEMPLATE = "{% for message in messages %}{{ message['role'] | capitalize }}{% if message['role'] == 'user' %}{{ '[INST] ' }}{{ '
```

classification with Naive Bayes an...

Exploring the Nave Bayes and Logistic regression classifiers, discussing the...

May 16, 2020 4

Wikipedia to Answer Open Domai...

This is a brief summary of my understanding on the DrQA system by Danqi Chen et.al.

Mar 10, 2021 3

Rajath Nag Nagaraj (Raj)

Deep Dive into Flights dataset using Tableau.

Flight delays are one of the most common things in airline domain. The reasons could...

Jul 4, 2020 1

Rajath Nag Nagaraj (Raj)

Analysis of Variance—ANOVA

ANOVA is used to figure out if there is a significant difference in the several group...

Aug 24, 2021 1

See all from Rajath Nag Nagaraj (Raj)

Recommended from Medium

In Towards AI by Rohan Mistry

The Complete LLM Fine-Tuning Guide: From Beginner to...

Stop Training Models from Scratch. Here's How to Fine-Tune Any LLM on Your Own...

Dec 11, 2025 108

In AI Advances by Delanoe Pirard

SLM vs LLM: Why 'Bigger is Better' is Dead in 2026

Gartner predicts 3x more SLM usage than LLMs by 2027. Bayer gained +40% accuracy...

6d ago 586 9

In TheUltimateInterviewHack by Sarthak Dargan

LoRA vs QLoRA: Why I Ditched Traditional Fine-tuning (Part 1)

Go Past the Paywall...

Aug 10, 2025 7

In Generative AI by Piyush Agnihotri

RAG From Scratch: Why LLMs Still Need RAG

Explore the limits of long context windows and understand why RAG remains critical fo...

5d ago 297 2

In Self-Supervised Learning by Frederik vom Lehn

Understanding SIGLIP, the more efficient vision encoder

Siglip [1] is an improved version of the contrastive CLIP image encoder.

Oct 4, 2025 54

Elven Kim

YOLOE to detect anything

🧠 Finally! YOLO-E Detects What I Describe—Not What It Was Trained On

Nov 27, 2025 43 2

See more recommendations